

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji
(An Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Accredited with 'A+' Grade by NAAC Department of Computer Science
and Engineering (Artificial Intelligence & Machine Learning) 2025-2026

A

PROJECT REPORT

ON

“THE EVENT MANAGEMENT SYSTEM”

Under the guidance

of

Prof. N.K.Mullani



Promoting Excellence in
Teaching, Learning & Research

Developed By :

1.Snehal Malkar 24UAM064

2.Siddhi Dabhade 24UAM020

CERTIFICATE

“EVENT MANAGEMENT SYSTEM”

This is to certify that the project entitled “**Event Management System**” has been successfully completed, in partial fulfillment of the requirements for the award of the degree in CSE(AIML) from DKTE during the academic year 2025-26. This project work has been carried out under the guidance and supervision of N.K.Mullani mam. The work embodied in this project is original and has not been submitted previously for the award of any other degree or diploma.

This is to certify that,

- | | |
|-------------------|----------|
| 1. Snehal Malkar | 24UAM064 |
| 2. Siddhi Dabhade | 24UAM020 |

Prof. N.K.Mullani
[Project Guide]

Prof. Dr. S.K. Shirgave
[Head of Department]

Date:
Place: Ichalkaranji

ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to our respected guide Prof. N. K. Mullani for giving us the valuable opportunity to undertake the project titled “Event Management System.” His guidance, motivation, and constructive suggestions throughout the development of this project were extremely helpful. His support not only helped us complete this project successfully but also enhanced our understanding of programming concepts and their real-world applications. We are highly thankful to our institution for providing the necessary infrastructure, resources, and a supportive academic environment that made the completion of this project possible. The knowledge gained through lectures, practical sessions, and academic activities played a significant role in shaping this project and improving our technical skills.

We would also like to express our sincere thanks to all our friends and classmates who helped us during the project work. Their cooperation, discussions, and valuable suggestions helped us solve problems and improve the overall quality of the project. Working together and sharing ideas made the learning experience more effective and enjoyable. We extend our heartfelt gratitude to our family members for their constant encouragement, patience, and moral support throughout the project. Their belief in us and continuous motivation helped us stay focused and complete our work with dedication and sincerity.

This project has been a great learning experience for us. It helped us understand the importance of planning, designing, and implementing a system using programming concepts. We gained practical knowledge of object-oriented programming, system design, and problem-solving techniques. This project also improved our logical thinking and ability to handle real-world problems effectively. We would like to thank everyone who directly or indirectly contributed to the successful completion of this project. Their support and encouragement have been invaluable, and we are truly grateful for their help.

DECLARATION

We the undersigned students of S.Y.C.S.E.(AIML) declare that the community engagement project work report entitled “**EVENT MANAGEMNT SYSTEM**” written and submitted under the guidance of Prof. N.K.Mullani is our original work. The empirical findings in this report are based on the data collected by us. The matter assimilated in this report is not reproduction from any readymade report.

Name

PRN

1. Snehal Malkar

24UAM064

2. Siddhi Dabhade

24UAM020

Date:

Place: Ichalkaranji

INDEX

Sr.No	CONTENTS	PAGE NO
1	Introduction	1
2	Objective	3
3	Literature Survey	4
4	Design Concept	5
5	Limitations	22
6	Future Scope	24
7	Algorithm	26
8	Output	30
9	Conclusion	52

INTRODUCTION

In today's modern and fast-paced world, organizing events has become an essential part of both personal and professional life. Events such as weddings, seminars, conferences, cultural programs, college functions, and corporate meetings require proper planning, coordination, and management. Traditionally, event management was carried out manually using registers, phone calls, and physical arrangements. This approach is not only time-consuming but also prone to errors, miscommunication, and poor data management. To overcome these challenges, the Event Management System has been developed as a digital solution to simplify and automate the entire process of event planning and management.

Event Management System is a software application designed to efficiently manage various aspects of events, including event creation, scheduling, booking, user management, and report generation. It provides a centralized platform where all event-related information can be stored, accessed, and updated easily. This system reduces manual effort and ensures better accuracy, speed, and organization of data. It is developed using the C++ programming language by applying Object-Oriented Programming (OOPs) concepts such as classes, objects, encapsulation, abstraction, inheritance, and polymorphism, which help in designing a structured, modular, and reusable system. Each entity in the system, such as Event, User, Admin, Booking, Review, and Report, is represented as a class, making the system easy to manage and extend.

The system is capable of handling different types of events including technical, non-technical, and college events. Technical events include activities such as coding competitions, hackathons, robotics competitions, paper presentations, and workshops, which require proper scheduling and participant management. Non-technical events include cultural and entertainment activities such as dance, singing, drama, sports, and quiz competitions that focus on creativity and engagement. College events include large-scale functions such as annual gatherings, fresher's parties, farewell events, seminars, and conferences that require efficient coordination and resource management.

The primary purpose of this system is to provide a user-friendly interface for both administrators and users. The administrator has complete control over the system and is responsible for managing all events by adding new events, updating existing details, deleting events, and viewing all records. Users can register themselves, log in securely, search for events based on their preferences, and book events according to their requirements. This interaction between administrators and users makes the system dynamic and efficient.

The system consists of multiple integrated modules such as admin, user, event, booking, review, and report modules, each performing specific functions that contribute to the overall system performance. It ensures that all event details such as name, type, budget, location, and date are properly stored and managed, while booking records are maintained accurately. Users are also able to provide feedback and ratings, which helps in improving the quality of services. The system generates reports that summarize key information such as the number of events, bookings, and users.

One of the major advantages of this system is that it saves time and significantly reduces manual work. Users can access all necessary information in one place without the need to contact multiple

sources, and they can complete the booking process quickly and efficiently. The system minimizes human errors and ensures that data is stored securely and can be retrieved whenever needed. It is capable of handling multiple users and events at the same time, making it flexible and scalable for both small and large-scale applications.

Security is an important aspect of the system, as user data is protected and only authorized users are allowed access to specific features. The system also focuses on providing a simple and easy-to-use interface so that even users with basic computer knowledge can operate it without difficulty. Smooth navigation allows users to search events, make bookings, and submit reviews with ease.

The development of this Event Management System provides practical exposure to software development concepts such as data structures, object-oriented programming in C++, and system design. It demonstrates how technology can be used to solve real-world problems effectively and helps in improving programming and analytical skills.

In conclusion, the Event Management System using OOPs in C++ is an efficient and reliable solution for managing technical, non-technical, and college events in a systematic manner. It simplifies event planning, reduces workload, improves accuracy, and enhances user satisfaction, making it a valuable tool for colleges, organizations, and event management companies.

OBJECTIVE

1. Efficient Event Handling

To allow the admin to add, view, search, and delete events easily without manual effort.

2. User Management

To provide secure user registration and login functionality so that only authorized users can access the system.

3. Event Booking System

To enable users to book events and manage their bookings without duplication.

4. Data Storage and Retrieval

To store all event, user, and booking data in files and retrieve it whenever required.

5. Avoid Duplicate Entries

To prevent duplicate event IDs, usernames, and bookings.

6. Review System

To allow users to give feedback/reviews only after attending (booking) an event.

7. Report Generation

To generate reports showing total events, users, and bookings for analysis.

8. User-Friendly Interface

To provide a simple menu-driven interface for easy interaction.

LITERATURE SURVEY

A literature survey is an important part of any project that involves studying existing systems, research work, and technologies related to the project topic. In the field of event management, various systems and methods have been developed to simplify the process of organizing and managing events. Earlier, event management was carried out manually using registers, phone calls, and physical arrangements. This traditional approach was time-consuming, less efficient, and prone to errors such as data loss, duplication, and miscommunication.

With the advancement of technology, computerized event management systems have been introduced to overcome these problems. These systems automate tasks such as event scheduling, participant registration, booking management, and report generation. Many institutions and organizations use web-based or desktop applications to manage different types of events efficiently. These systems provide centralized data storage, easy access to information, and improved accuracy.

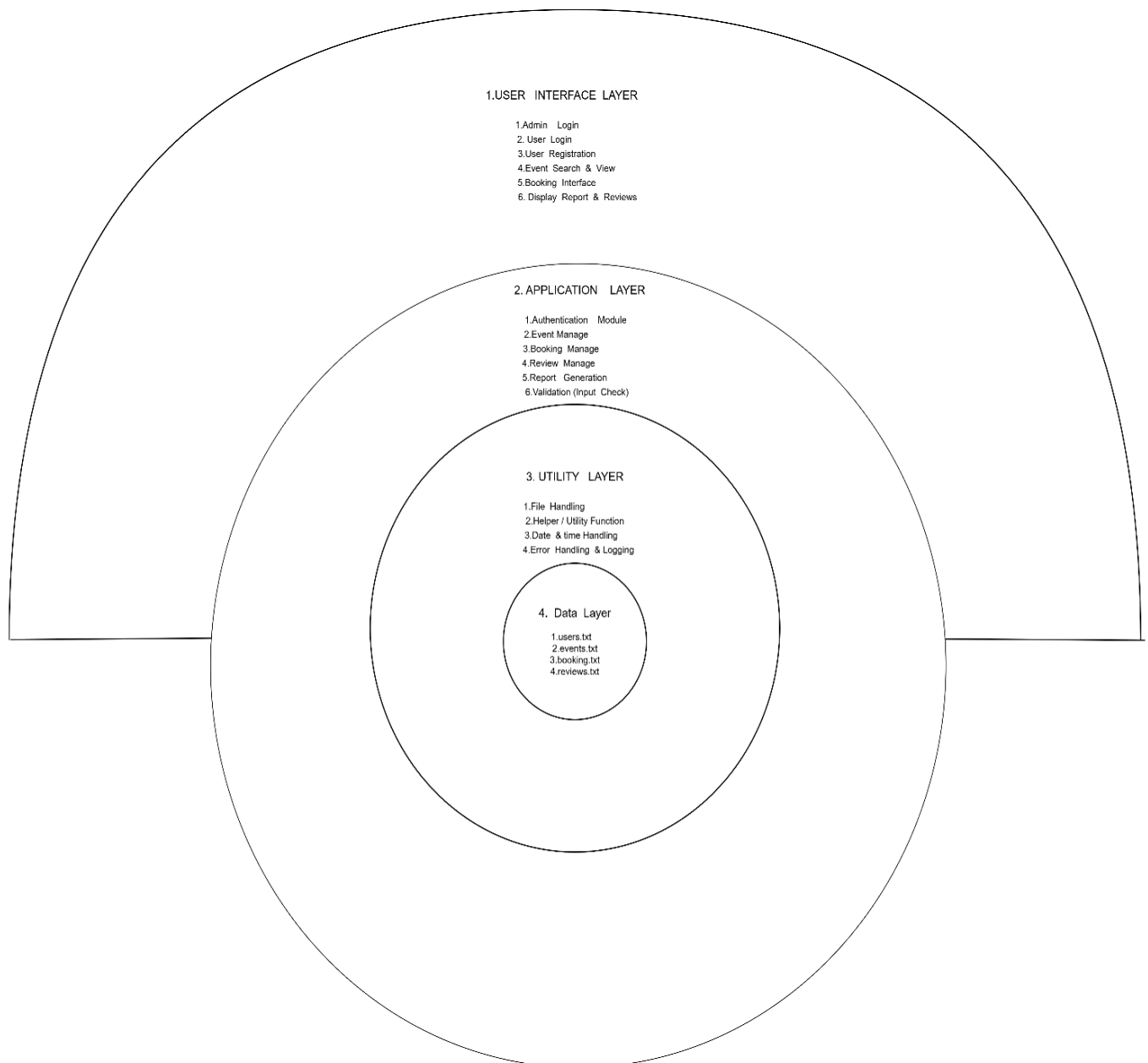
Existing event management systems support different types of events, including technical, non-technical, and college events. They allow administrators to create and manage events, while users can register, view event details, and book events. Some advanced systems also provide additional features such as online payments, notifications, feedback systems, and real-time updates, which enhance user experience and system functionality.

From a technical perspective, these systems are developed using programming languages such as Java, Python, and C++. Among these, C++ using Object-Oriented Programming (OOPs) concepts is widely used for building structured and efficient systems. Concepts like classes, objects, encapsulation, inheritance, and polymorphism help in designing modular, reusable, and secure applications. However, many existing systems still have certain limitations. Some systems are complex and difficult to use, while others lack user-friendly interfaces or proper data security. In some cases, systems depend heavily on internet connectivity or do not support all types of events effectively. These limitations highlight the need for a simple, efficient, and secure event management system.

The proposed Event Management System aims to overcome these drawbacks by providing a user-friendly interface, efficient data handling, and support for multiple types of events such as technical, non-technical, and college events. It is developed using OOPs concepts in C++, ensuring better performance, modularity, and scalability. The system includes essential modules such as admin, user, event, booking, and report management to handle all event-related activities effectively.

In conclusion, the literature survey shows that although many event management systems are available, there is still a need for a more efficient, user-friendly, and secure system. The proposed system fulfills these requirements and provides a better solution for modern event management.

ARCHITECTURE DIAGRAM



LAYERED ARCHITECTURE

ARCHITECTURE DIAGRAM EXPLANATION

The given diagram represents a Layered Architecture Design of an Event Management System. This architecture divides the system into multiple layers, where each layer performs a specific function and communicates only with adjacent layers.

The main objective of using a layered architecture is to achieve:

- Separation of concerns
- Better maintainability
- High scalability
- Improved security and modularity

The system is divided into four main layers:

1. User Interface Layer
2. Application Layer
3. Utility Layer
4. Data Layer

1. User Interface Layer (Presentation Layer)

This is the outermost layer of the system and acts as an interface between the user and the system. All interactions between the user (Admin/User) and the system occur through this layer.

Features:

- Provides graphical or textual interface (forms, menus, screens)
- Accepts user inputs
- Displays output/results

Functional Components:

1. **Admin Login** – Allows admin to log into the system
2. **User Login** – Allows registered users to access their accounts
3. **User Registration** – Enables new users to create accounts
4. **Event Search & View** – Users can search and browse events
5. **Booking Interface** – Allows users to book events
6. **Display Reports & Reviews** – Shows reports (for admin) and reviews (for users)

Role:

- Sends user requests to the Application Layer

- Receives processed results and displays them
- Does not contain business logic (only presentation)

2. Application Layer (Business Logic Layer)

This layer is the core of the system, where all business logic and decision-making processes are implemented.

Modules:

1. **Authentication Module**
 - Verifies user/admin credentials
 - Ensures secure access
2. **Event Management Module**
 - Add, update, delete, and view events
 - Controlled mainly by admin
3. **Booking Management Module**
 - Handles booking operations
 - Links users with events
4. **Review Management Module**
 - Manages user reviews and ratings
 - Improves service quality
5. **Report Generation Module**
 - Generates summaries like total users, bookings, events
6. **Validation (Input Checking)**
 - Ensures correct and valid user input
 - Prevents errors and invalid data entry

Role:

- Processes requests received from UI
- Applies system rules and logic
- Communicates with Utility Layer and Data Layer

3. Utility Layer

The Utility Layer provides supporting services required by the Application Layer. It contains reusable functions that simplify development and reduce redundancy.

Functions:

1. **File Handling**
 - Reading and writing data to files
2. **Helper / Utility Functions**
 - Common reusable functions used across modules

3. **Date & Time Handling**

- Manages event dates, booking dates, etc.

4. **Error Handling & Logging**

- Detects and handles system errors
- Maintains logs for debugging and tracking

Role:

- Supports the Application Layer
- Improves code efficiency and reusability
- Handles low-level operations

4. **Data Layer (Storage Layer)**

This is the innermost layer responsible for storing and managing system data.

Data Storage Files:

- **users.txt** → Stores user account information
- **events.txt** → Stores event details
- **booking.txt** → Stores booking records
- **reviews.txt** → Stores user reviews

Role:

- Maintains persistent data
- Performs data retrieval and storage operations
- Ensures data consistency

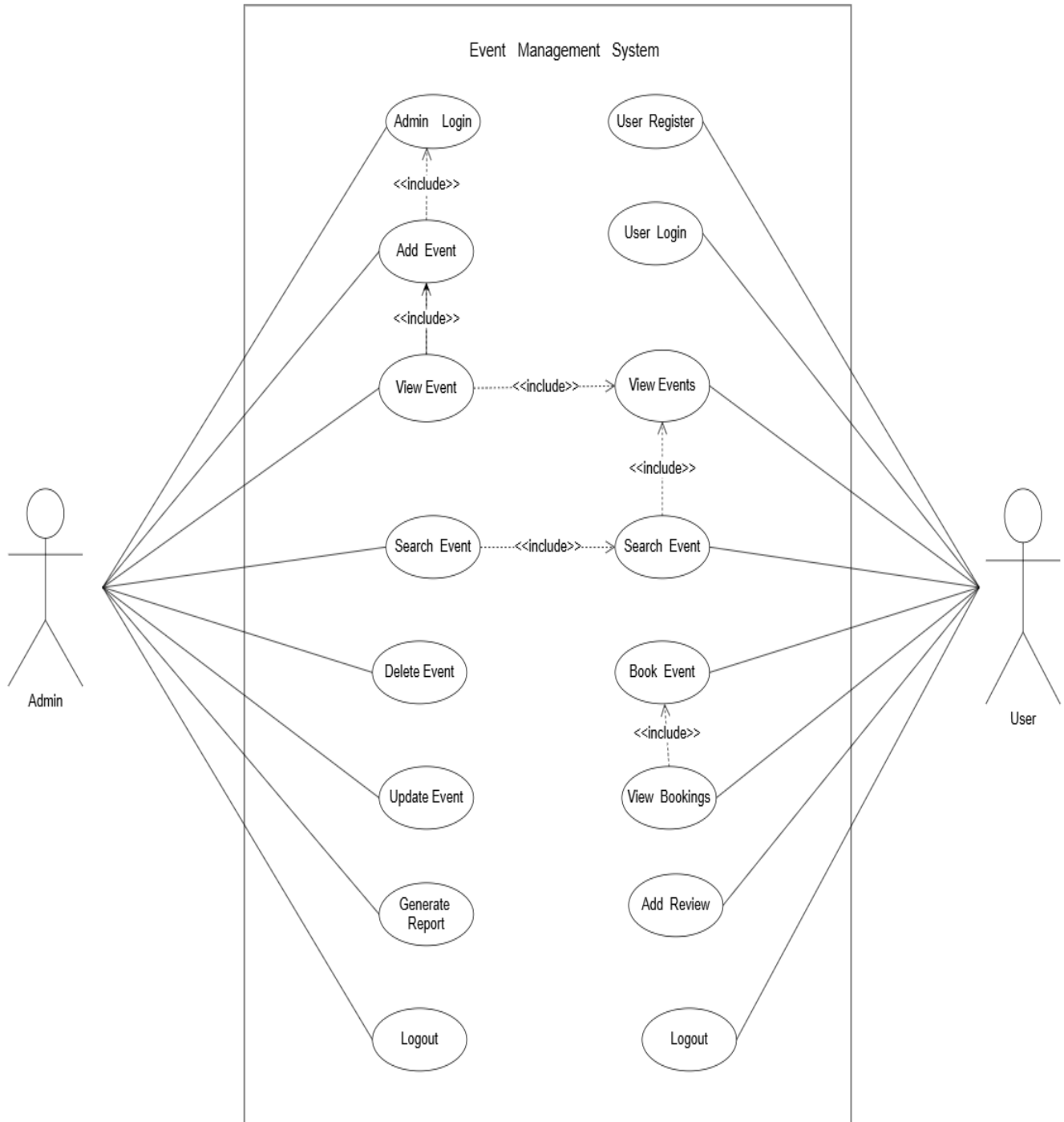
System Working Flow

1. The user interacts with the system through the User Interface Layer
2. The request is passed to the Application Layer
3. The Application Layer processes logic and validates input
4. It uses the Utility Layer for supporting operations
5. Data is stored/retrieved from the Data Layer
6. The processed result is returned back to the UI

Conclusion

The layered architecture diagram clearly demonstrates how the Event Management System is structured in a well-organized manner. Each layer performs a dedicated function and interacts systematically with other layers. This design improves system performance, maintainability, and scalability while following standard software engineering principles.

USE CASE DIAGRAM



USE-CASE DIAGRAM EVENT MANAGEMENT SYSTEM

USE CASE DIAGRAM EXPLANATION

The Use Case Diagram of the Event Management System illustrates how different actors interact with the system to perform various operations. It represents the functional requirements of the system and defines the roles and responsibilities of each user.

The diagram consists of a system boundary (Event Management System) which encloses all the use cases, and two primary actors:

- **Admin**
- **User**

These actors interact with the system through different use cases.

1. Admin Actor

The Admin is the central authority of the system who manages all event-related activities. The admin has complete access to system functionalities.

Admin Use Cases

1. Login

The Admin must log in using valid credentials to access the system. This is a mandatory step before performing any operation.

The <<include>> relationship shows that other actions depend on Login.

2. Add Event

The Admin can create and add new events into the system. This includes entering details such as:

1. Event Id
2. Event Name
3. Event Type
4. Event Location
5. Budget

Includes Login, ensuring only authenticated users can add events.

3. View Event

The Admin can view the list of all available events along with their details. It includes Add Event, which means events must exist before viewing.

4. Search Event

The Admin can search for specific events using filters such as:

1. Event name
2. Date
3. Type

Helps in quick access to specific data.

5. Delete Event

The Admin can delete events that are canceled or no longer needed. This helps in maintaining accurate and updated data.

6. Update Event

The Admin can update or modify existing event details like:

1. Date changes
2. Venue updates
3. Pricing modifications

7. Generate Report

The Admin can generate reports related to:

1. Events
2. Bookings
3. User participation

This feature is useful for analysis and decision-making.

8. Logout

After completing tasks, the Admin logs out of the system to ensure security.

2. User Actor

The User is the end-user who interacts with the system to explore and participate in events.

User Use Case

1. Register

A new user must register by providing personal details such as name, email, and password.

2. Login

The user logs into the system using registered credentials to access functionalities.

3. View Events

The user can view all available events in the system.

Includes Search Event for better navigation.

4. Search Event

The user can search events based on preferences like:

1. Location
2. Date
3. Type of event

5. Book Event

The user can book tickets for selected events.

Includes View Bookings, allowing the user to verify booking details.

6. View Bookings

The user can check details of booked events such as:

1. Booking ID
2. Event details
3. Date and time

7. Add Review

After attending an event, the user can provide feedback or reviews. This helps improve future events.

8. Logout

The user logs out after completing their activities.

Relationships in the Diagram

<<include>> Relationship

The diagram uses the <<include>> relationship, which indicates that one use case depends on another use case.

1. Add Event → includes Login
2. View Event → includes Add Event
3. View Events → includes Search Event
4. Book Event → includes View Bookings

This relationship improves modularity and avoids repetition in the system design.

System Boundary

The rectangle in the diagram represents the system boundary, which includes all functionalities of the Event Management System.

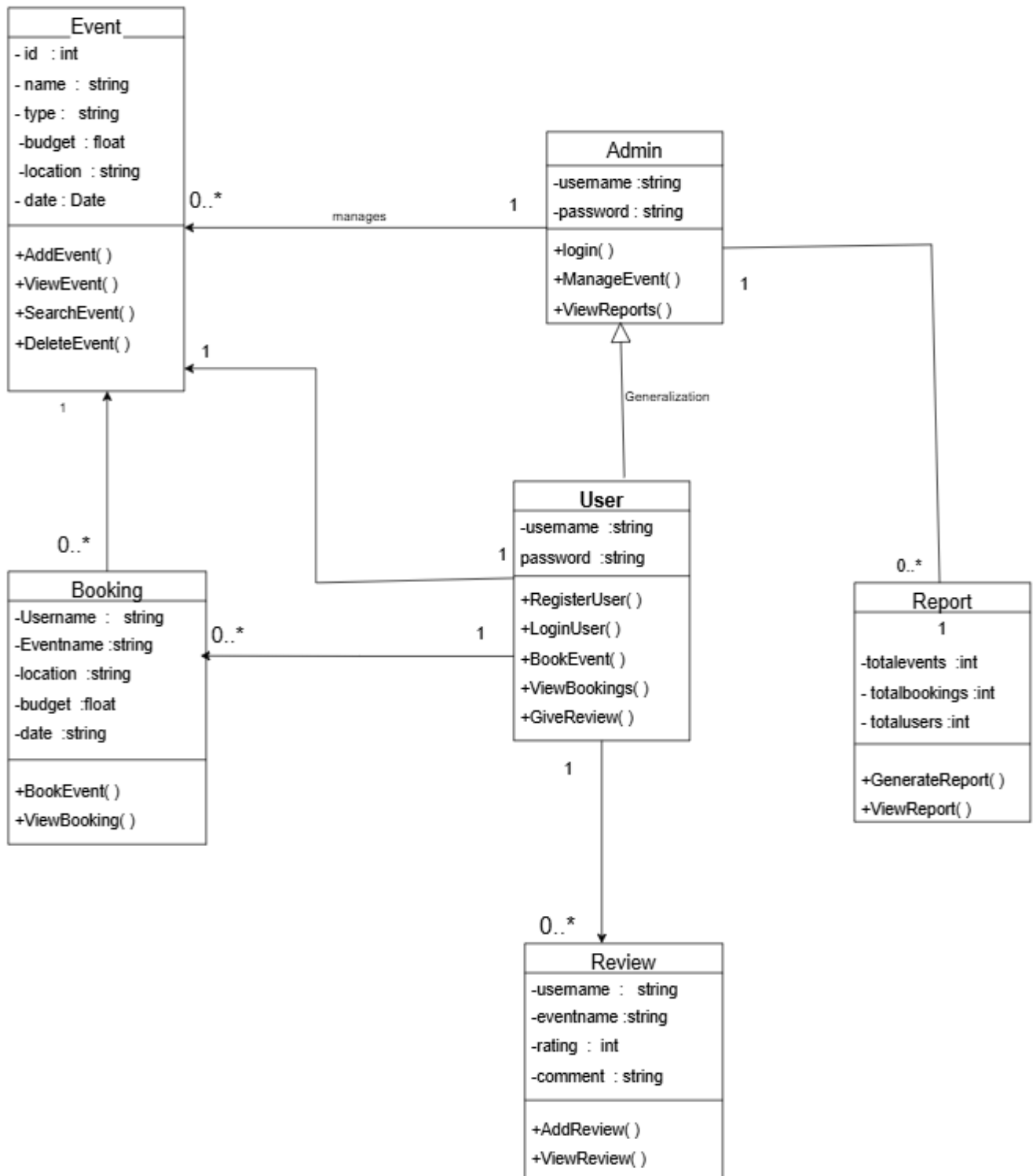
- Everything inside the boundary = system function
- Everything outside (actors) = external users interacting with the system

Conclusion

The Use Case Diagram clearly defines:

1. The interaction between Admin and User
2. The functionalities provided by the system
3. The dependencies between different operations

CLASS DIAGRAM



CLASS DIAGRAM EXPLANATION

1. Event Class

1. Represents all events in the system
2. Stores event details like ID, name, type, budget, location, and date
3. Attributes:
 - id : int → Unique identifier of the event
 - name : string → Name of the event
 - type : string → Type/category (e.g., concert, seminar)
 - budget : float → Cost allocated
 - location : string → Where it is held
 - date : Date → Event date
4. Methods:
 - AddEvent() → Adds a new event
 - ViewEvent() → Displays event details
 - SearchEvent() → Searches events
 - DeleteEvent() → Deletes an event
5. Relationship:
 - Managed by Admin
 - One event can have multiple bookings

2. User Class

1. Represents a general system user
2. Stores login details (username, password)
3. Methods:
 - RegisterUser() → Registers a new user
 - LoginUser() → User login
 - BookEvent() → Books an event
 - ViewBookings() → Shows booking history
 - GiveReview() → Adds review
4. Relationship:
 - One user can make multiple bookings
 - One user can give multiple reviews

3. Admin Class

1. Specialized class that inherits from User
2. Has additional system control features
3. Methods:
 - login() → Admin login

- ManageEvent() → Manage event operations
 - ViewReports() → View reports
4. Relationship:
- Admin manages events
 - Admin generates and views reports

4. Booking Class

1. Stores booking information of users
2. Attributes include username, event name, location, budget, and date
3. Methods:
 - BookEvent() → Creates booking
 - ViewBooking() → Displays booking details
4. Relationship:
 - Each booking is linked to one user and one event
 - One event can have many bookings

5. Review Class

1. Stores feedback given by users for events
2. Includes rating and comments
3. Methods:
 - AddReview() → Adds review
 - ViewReview() → Displays reviews
4. Relationship:
 - One user can give multiple reviews
 - Each review belongs to one event

6. Report Class

1. Generates summary of system data
2. Attributes:
 - totalEvents
 - totalBookings
 - totalUsers
3. Methods:
 - GenerateReport() → Creates reports
 - ViewReport() → Displays reports
4. Relationship:
 - Admin generates and views reports

7. Relationships in Diagram

1. Inheritance:

- Admin inherits from User

2. Associations:

- Admin ↔ Event (manages)
- User ↔ Booking (books events)
- Event ↔ Booking (has bookings)
- User ↔ Review (writes reviews)
- Admin ↔ Report (generates reports)

8. Multiplicity

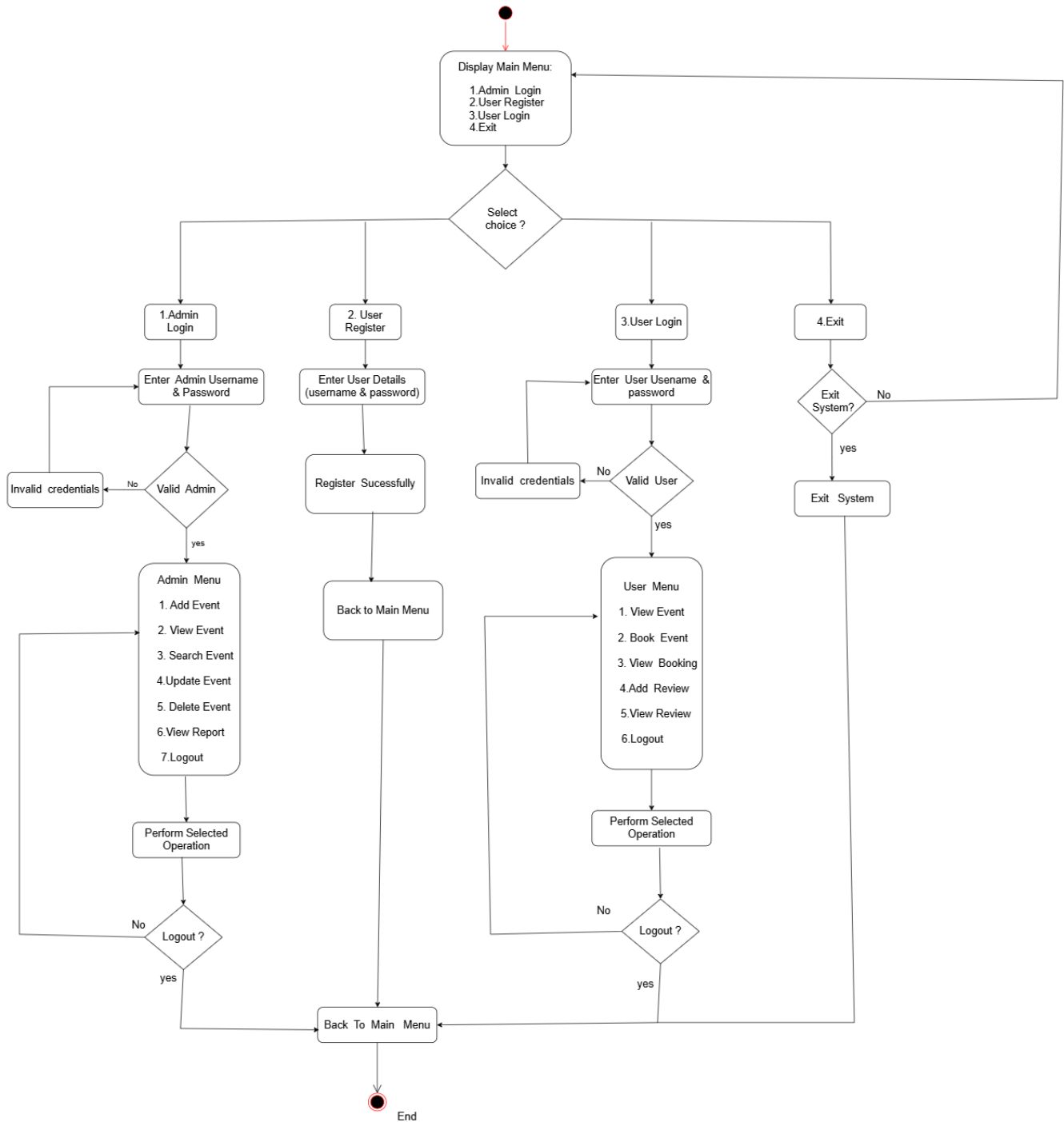
1. $1 \rightarrow 0..*$ (One-to-Many relationship):

- One user $\rightarrow$ many bookings
- One event $\rightarrow$ many bookings
- One user $\rightarrow$ many reviews
- One admin $\rightarrow$ many reports

9. Conclusion

- The diagram clearly defines system structure
- Supports event booking, reviewing, and management
- Follows object-oriented principles
- Ensures scalability, maintainability, and modular design

ACTIVITY DIAGRAM



ACTIVITY DIAGRAM EXPLANATION

The activity diagram of the Event Management System represents the complete workflow of the system, showing how different activities are performed by the Admin and User from system start to end. It clearly describes the sequence of operations, decision-making points, and flow of control.

1. Start of the System

The process begins with the initial node, which indicates the launching of the system. After starting, the system displays the Main Menu to the user.

2. Main Menu Options

The main menu provides four options:

1. Admin Login
2. User Registration
3. User Login
4. Exit

A decision node (Select Choice) is used to select the appropriate path based on the user's input.

3. Admin Login Process

If the Admin Login option is selected:

The admin enters username and password.

- The system verifies the credentials using the decision node "Valid Admin?".

If Valid:

- The admin accesses the Admin Menu, which includes:
 1. Add Event
 2. View Events
 3. Search Event
 4. Update Event
 5. Delete Event
 6. View Report
 7. Logout
- The admin performs required operations.
- The system asks for Logout:
 1. If No, continue operations

2. If Yes, return to Main Menu

If Invalid:

- The system displays Invalid Credentials.
- The admin must re-enter details.

4. User Registration

If the User Register option is selected:

- The user enters required details (username and password).
- The system displays “Registration Successful”.
- The user is redirected back to the Main Menu.

5. User Login Process

If the User Login option is selected:

- The user enters login credentials.
- The system checks “Valid User?”.

If Valid:

- The user accesses the User Menu, which includes:

1. View Events
2. Book Event
3. View Booking
4. Add Review
5. View Review
6. Logout

- The user performs actions.
- The system asks for Logout:
 1. If No, continue
 2. If Yes, go back to Main Menu

If Invalid:

- The system shows Invalid Credentials.
- The user retries login.

6. Exit Process

If the Exit option is selected:

- The system asks “Exit System?”
 1. If Yes, the system terminates
 2. If No, returns to Main Menu

7. End of the System

The process ends at the final node, indicating system termination.

8. Overall System Flow

The system follows a structured and logical flow:

- It starts with a menu-driven interface.
- Based on user selection, it branches into Admin, User, or Exit processes.
- It ensures authentication through validation checks.
- It allows users to perform operations repeatedly until they choose to logout.
- The system provides flexibility by returning to the Main Menu after each process.

9. Conclusion

The activity diagram effectively represents the working mechanism of the Event Management System by clearly illustrating all activities, decision points, and user interactions. It improves understanding of the system design, helps developers implement functionalities correctly, and serves as a useful documentation tool. The diagram ensures clarity, proper flow control, and efficient handling of user operations.

LIMITATIONS OF PROJECT

1. No Graphical User Interface (GUI)

System works only on console (text-based), not user-friendly.

2. Basic Security

- Admin login is hardcoded
- Passwords are stored in plain text (not encrypted)

3. No Database Integration

Uses text files instead of database (MySQL, etc.), so:

- Data handling is slow for large records
- No proper data relationships

4. Limited Data Validation

- Date validation is basic
- Invalid formats may still be accepted

5. No Multi-user Support

Only one user can use system at a time (no concurrency handling)

6. File Handling Issues

- Data may corrupt if file is modified manually
- No backup or recovery system

7. Space Handling Problem

Input like event name/location with spaces may not work properly

8. No Update Feature

Cannot update/edit existing event details

9. Limited Search Functionality

Search only by Event ID (not by name, date, etc.)

10.No Payment Integration

Booking system does not handle real payments

11.No Notification System

Users do not get reminders for upcoming events

12.No Role Management

Only Admin and User roles, no advanced permissions

13.No Data Encryption

Sensitive data is not protected

14.Scalability Issues

Not suitable for large-scale real-world applications

15.No Error Logging System

System does not track errors or crashes

FUTURE SCOPE

1. Graphical User Interface (GUI)

Develop a user-friendly interface using Java, Python, or web technologies.

2. Database Integration

Replace text files with databases (MySQL, MongoDB) for better performance and data management.

3. Online Web-Based System

Convert into a web application so users can access it anytime, anywhere.

4. Mobile Application

Create Android/iOS app for easy booking and event management.

5. Secure Authentication System

- Password encryption
- OTP verification
- Role-based login system

6. Online Payment Integration

Add payment gateways (UPI, cards, net banking) for real booking system.

7. Advanced Search & Filters

Search events by name, date, location, type, budget, etc.

8. Event Update Feature

Allow admin to edit/update event details.

9. Notification System

Email/SMS alerts for:

- Booking confirmation
- Event reminders

10. AI-Based Recommendations

Suggest events based on user interests and previous bookings.

11. Report & Analytics Dashboard

Graphs and statistics for admin (bookings, revenue, popular events).

12. Multi-user & Real-time Access

Support multiple users simultaneously using server-based system.

13.Cloud Storage Integration

Store data securely on cloud platforms (AWS, Google Cloud).

14.Review & Rating System Enhancement

Add star ratings, comments, and feedback analysis.

15.Multilingual Support

Support different languages for wider usability.

ALGORITHM

1. Start the Program

1. Begin execution of the program.
2. Display the Main Menu:
 - Admin Login
 - User Registration
 - User Login
 - Exit

2. Admin Login Process

1. Ask the admin to enter username and password.
2. Check if credentials match predefined admin data.
3. If valid:
 - Display Admin Menu:
 1. Add Event
 2. View Events
 3. Search Event
 4. Delete Event
 5. Generate Report
 6. Clear All Data
 7. Exit
4. If invalid:
 - Display "Invalid Admin Login".

3. Add Event

1. Input Event ID and Event Name.
2. Check:
 - If Event ID or Name already exists → Reject.
3. Input:
 - Type, Budget, Location, Date.
4. Validate:
 - Date must be in future.
5. If valid:
 - Store event details in events.txt.
6. Display "Event Added Successfully".

4. View Events

1. Open events.txt.
2. Read all event records.
3. Display in tabular format:

- ID, Name, Type, Budget, Location, Date.

5. Search Event

1. Input Event ID.
2. Search in events.txt.
3. If found:
 - Display event details.
4. Else:
 - Display "Event Not Found".

6. Delete Event

1. Input Event ID.
2. Read all events from file.
3. Copy all records except selected ID into a temporary file.
4. Replace original file with temporary file.
5. Display "Event Deleted".

7. Generate Report

1. Count total:
 - Events from events.txt
 - Users from users.txt
 - Bookings from bookings.txt
2. Display:
 - Total Events
 - Total Users
 - Total Bookings

8. Clear All Data

1. Ask for confirmation (Y/N).
2. If Yes:
 - Delete all data from:
 - events.txt
 - users.txt
 - bookings.txt
 - reviews.txt
3. Display "All Data Cleared".

9. User Registration

1. Input username and password.
2. Check if username already exists.
3. If not:

- Save in users.txt.
4. Display "Registration Successful".

10. User Login

1. Input username and password.
2. Verify from users.txt.
3. If valid:
 1. Open User Menu:
 - View Events
 - Book Event
 - Delete Booking
 - Add Review
 - Exit
4. Else:
 1. Display "Invalid Login".

11. Book Event

1. Input Event ID.
2. Check:
 - Event exists
 - User has not already booked
3. If valid:
 - Store booking in bookings.txt.
4. Display "Booking Successful".

12. Delete Booking

1. Input Event ID.
2. Search booking in bookings.txt.
3. Remove matching record using temporary file.
4. Display:
 - "Booking Deleted" OR "Booking Not Found".

13. Add Review

1. Input Event ID.
2. Check:
 - Event exists
 - User has booked the event
 - User has not already reviewed
3. Input review text.
4. Save in reviews.txt.
5. Display "Review Added".

14. Exit Program

1. If user selects Exit:

Terminate the program.

```

24UAM064 Project

EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'
  TIMELINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Win-10\Documents\24UAM064 Project> g++ event.cpp
PS C:\Users\Win-10\Documents\24UAM064 Project> ./a.exe

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 1
Admin Username: admin
Password: 1234

----- Admin MENU -----
1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 101
Enter Name: codechef
Enter Type: technical
Enter Budget: 8000
Enter Location: kolhapur
Enter Date (dd/mm/yyyy): 06/08/2026
Event Added!

----- Admin MENU -----
1.Add Event
2.View Event

```

Visual Studio Code interface showing the Event Management System project. The Explorer sidebar on the left shows the file structure with 'event.cpp' selected. The Terminal window on the right displays the program's output.

```

24UAM064 Project

EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 102
Enter Name: fashionshow
Enter Type: nontechnical
Enter Budget: 12000
Enter Location: sangli
Enter Date (dd/mm/yyyy): 14/06/2026
Event Added!

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 103
Enter Name: hackathon
Enter Type: technical
Enter Budget: 23000
Enter Location: miraj
Enter Date (dd/mm/yyyy): 08/07/2026
Event Added!

```

```

24UAM064 Project
EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'
  TIMELINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 104
Enter Name: championcup
Enter Type: sports
Enter Budget: 9000
Enter Location: tardal
Enter Date (dd/mm/yyyy): 04/05/2026
Event Added!

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 105
Enter Name: salesguru
Enter Type: nontechnical
Enter Budget: 23000
Enter Location: pune
Enter Date (dd/mm/yyyy): 23/07/2026
Event Added!

```

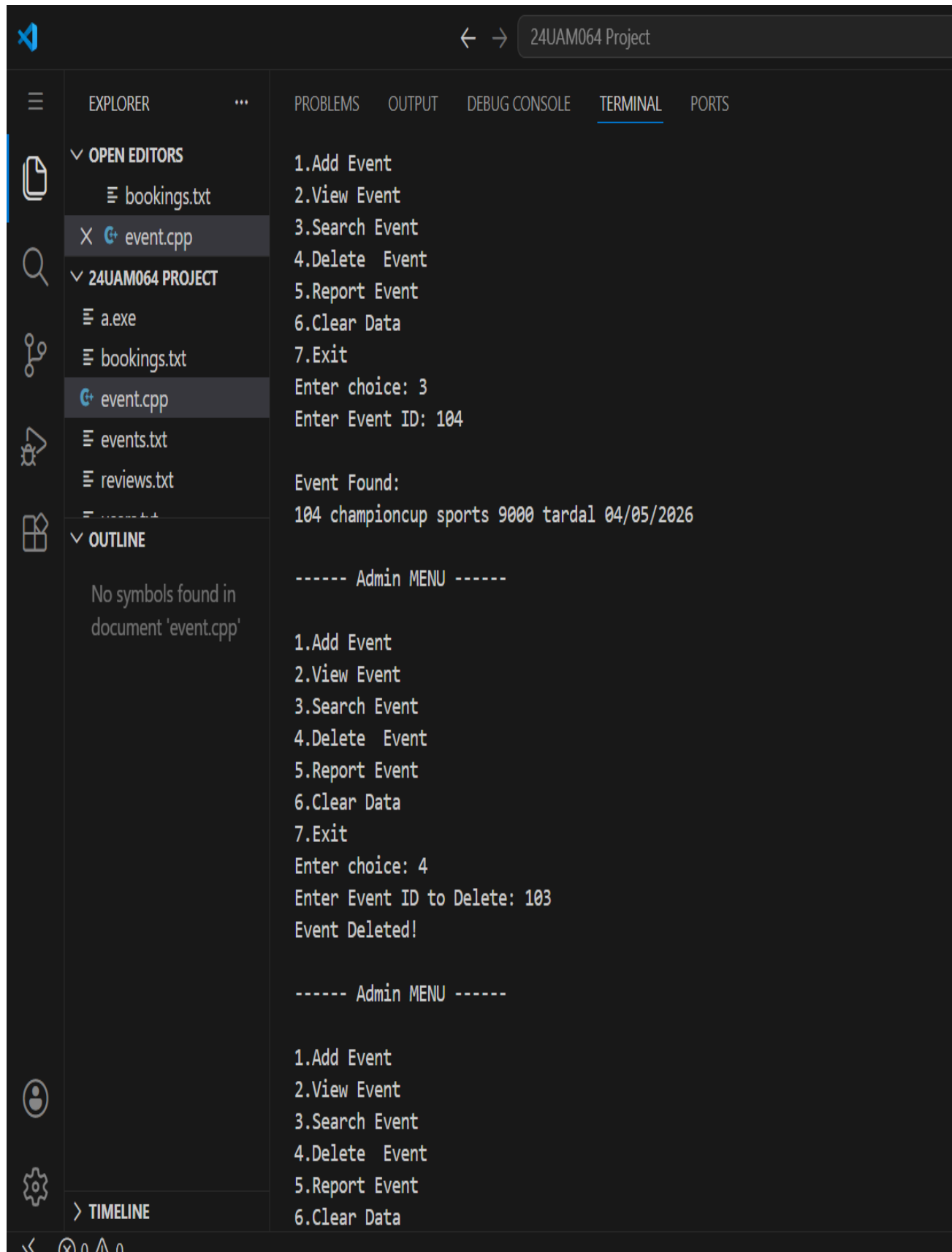
```

24UAM064 Project
1
EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'
  TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 1
Enter Event ID: 106
Enter Name: rangtarang
Enter Type: cultural
Enter Budget: 10000
Enter Location: ichalkarnji
Enter Date (dd/mm/yyyy): 09/03/2025
Enter future date only!

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 2
      ID      NAME      TYPE      BUDGET  LOCATION  DATE
      101  codechef  technical    8000   kolhapur  06/08/2026
      102  fashionshow  nontechnical 12000   sangli    14/06/2026
      103   hackathon  technical   23000   miraj     08/07/2026
      104  championcup    sports     9000   tardal    04/05/2026
      105  salesgurunon  nontechnical 23000    pune     23/07/2026

```



Visual Studio Code interface showing the Explorer, Problems, Output, Debug Console, and Terminal panels. The Explorer panel shows the project structure with 'event.cpp' selected. The Terminal panel shows the execution of the program, displaying a menu and user input.

EXPLORER

- OPEN EDITORS
 - bookings.txt
 - event.cpp
- 24UAM064 PROJECT
 - a.exe
 - bookings.txt
 - event.cpp
 - events.txt
 - reviews.txt
- OUTLINE
 - No symbols found in document 'event.cpp'
- TIMELINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 3
Enter Event ID: 104

Event Found:
104 championcup sports 9000 tardal 04/05/2026

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 4
Enter Event ID to Delete: 103
Event Deleted!

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
  
```

```

24UAM064 Project
EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'
  TIMELINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

5.Report Event
6.Clear Data
7.Exit
Enter choice: 2

      ID      NAME      TYPE      BUDGET      LOCATION      DATE
    101  codechef  technical      8000      kolhapur      06/08/2026
    102  fashionshow  nontechnical  12000      sangli      14/06/2026
    104  championcup    sports      9000      tardal      04/05/2026
    105  salesgurunontechnical  23000      pune      23/07/2026

----- Admin MENU -----

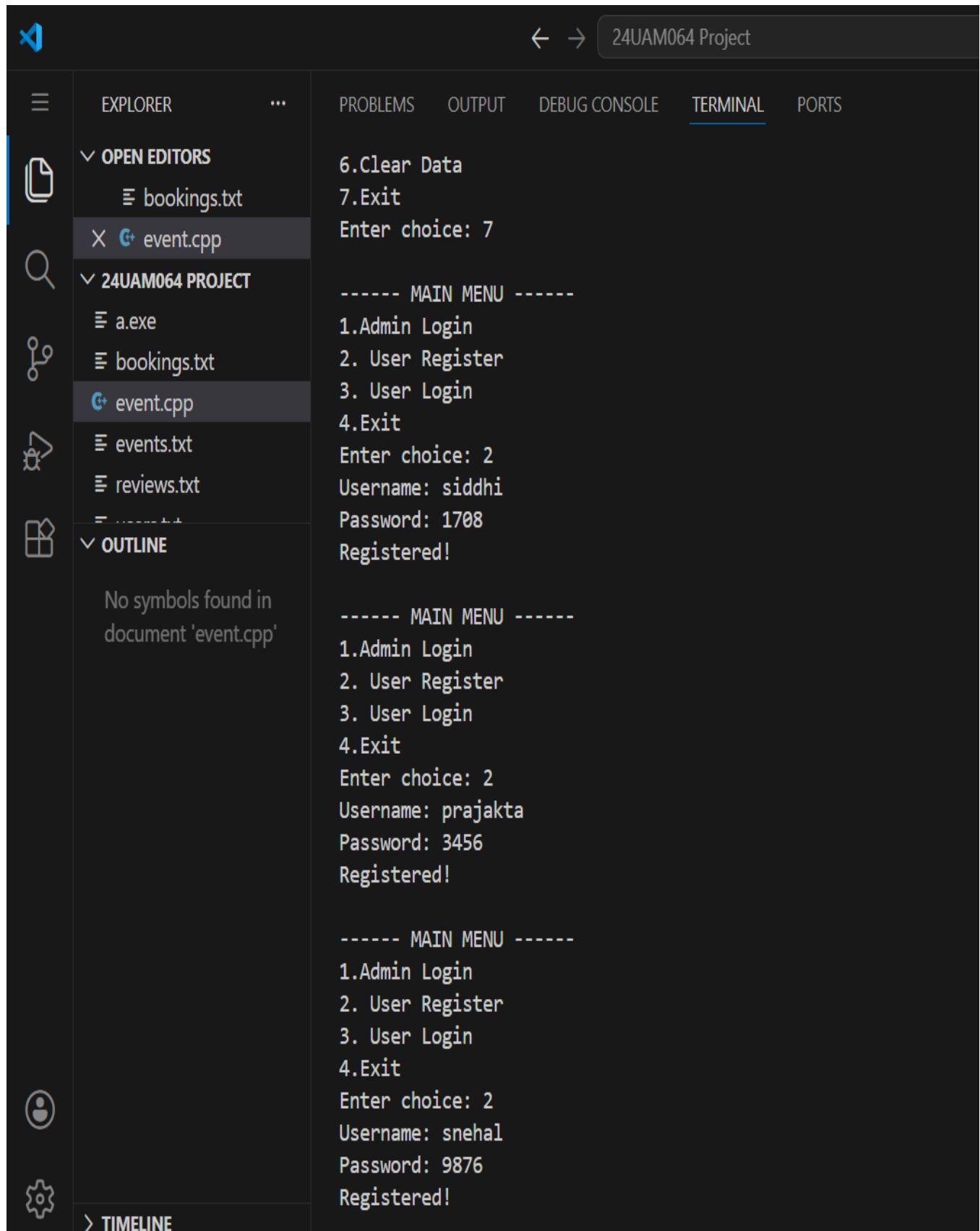
1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 5

===== REPORT =====
Events: 4
Users: 0
Bookings: 0

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data

```

Visual Studio Code interface showing the Explorer, Output, and Terminal panels. The Explorer panel displays the project structure for '24UAM064 PROJECT', including files like 'bookings.txt', 'event.cpp', 'a.exe', 'events.txt', and 'reviews.txt'. The Output panel shows the execution of the program, displaying the main menu and user registration details for three different users: siddhi, prajakta, and snehal.

```
6.Clear Data
7.Exit
Enter choice: 7

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: siddhi
Password: 1708
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: prajakta
Password: 3456
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: snehal
Password: 9876
Registered!
```

24UAM064 Project

EXPLORER

OPEN EDITORS

- bookings.txt
- event.cpp

24UAM064 PROJECT

- a.exe
- bookings.txt
- event.cpp
- events.txt
- reviews.txt

OUTLINE

No symbols found in document 'event.cpp'

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

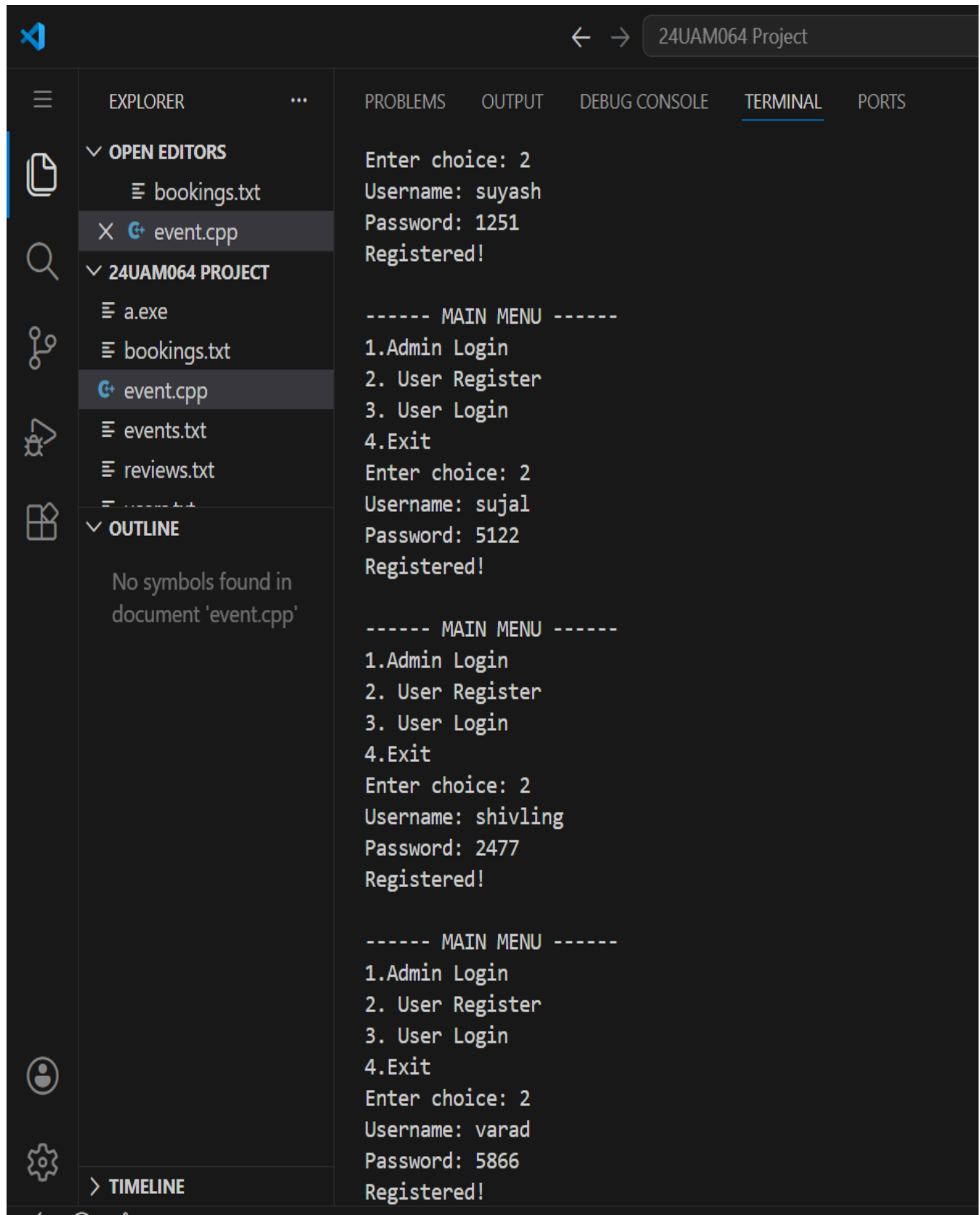
```
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: saniya
Password: 6688
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: anjali
Password: 6677
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: indranil
Password: 4465
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
```

> TIMELINE



Visual Studio Code interface showing the Explorer, Output, and Terminal panels. The Explorer panel displays the project structure for '24UAM064 PROJECT', including files like 'a.exe', 'bookings.txt', 'event.cpp', 'events.txt', and 'reviews.txt'. The Output panel shows the execution of the program, displaying the main menu and user registration details for three users: suyash, sujal, and shivling.

```
Enter choice: 2
Username: suyash
Password: 1251
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: sujal
Password: 5122
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: shivling
Password: 2477
Registered!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 2
Username: varad
Password: 5866
Registered!
```

The screenshot shows the Visual Studio Code interface with the following components:

- Explorer Panel:**
 - OPEN EDITORS: bookings.txt, event.cpp (selected)
 - 24UAM064 PROJECT:
 - a.exe
 - bookings.txt
 - event.cpp (selected)
 - events.txt
 - reviews.txt
- Outline Panel:** No symbols found in document 'event.cpp'
- Terminal Panel:**

```

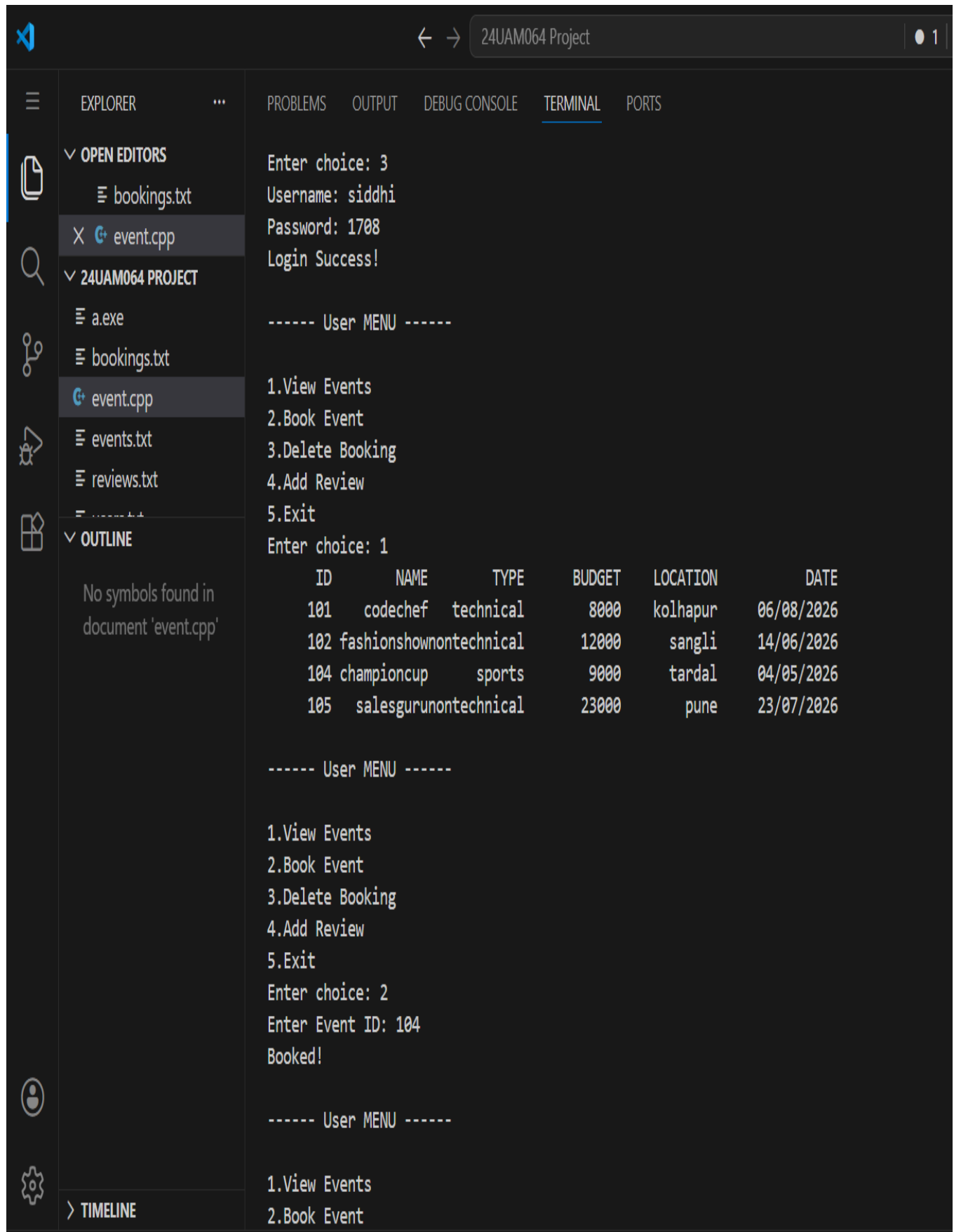
----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 3
Username: siddhi
Password: 1708
Login Success!

----- User MENU -----
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 5

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 3
Username: shreya
Password: 1111
Invalid!

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit

```



Visual Studio Code window showing the '24UAM064 Project' with the 'event.cpp' file open. The terminal displays the program's output, including a login success message and a menu-driven interface for event management.

```

Enter choice: 3
Username: siddhi
Password: 1708
Login Success!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 1

  ID      NAME      TYPE      BUDGET  LOCATION  DATE
  101    codechef  technical    8000    kolhapur  06/08/2026
  102  fashionshow  nontechnical 12000    sangli    14/06/2026
  104  championcup    sports     9000    tardal    04/05/2026
  105  salesgurunon  nontechnical 23000    pune      23/07/2026

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 104
Booked!

----- User MENU -----

1.View Events
2.Book Event

```

The screenshot displays the Visual Studio Code interface for the '24UAM064 Project'. The Explorer panel on the left shows the file structure with 'event.cpp' selected. The Outline panel below it indicates 'No symbols found in document 'event.cpp''. The Terminal panel on the right shows the execution output of the program.

```

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 102
Booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 105
Booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 3
Enter Event ID to cancel: 105
Booking Deleted!

----- User MENU -----

```

The screenshot displays the Visual Studio Code interface for the '24UAM064 Project'. The Explorer panel on the left shows the project structure with files like 'bookings.txt', 'event.cpp', 'a.exe', 'events.txt', and 'reviews.txt'. The Output panel shows the execution of the program, displaying a menu and user input. The Terminal panel shows the same output as the Output panel.

```

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 102
Enter Review: excellent
Review Added!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 104
Enter Review: verygood
Review Added!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 5

----- MAIN MENU -----
1.Admin Login
  
```

```

24UAM064 Project
1
1
EXPLORER
OPEN EDITORS
bookings.txt
event.cpp
24UAM064 PROJECT
a.exe
bookings.txt
event.cpp
events.txt
reviews.txt
OUTLINE
No symbols found in document 'event.cpp'
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 3
Username: suyash
Password: 1251
Login Success!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 1

  ID      NAME      TYPE      BUDGET  LOCATION  DATE
  101    codechef  technical    8000    kolhapur  06/08/2026
  102  fashionshownontechical  12000    sangli   14/06/2026
  104    championcup    sports    9000    tardal   04/05/2026
  105    salesgurunontechical  23000    pune     23/07/2026

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 101
Booked!

```


The screenshot shows the Visual Studio Code interface with the following components:

- Explorer Panel:**
 - OPEN EDITORS:** bookings.txt, event.cpp (selected).
 - 24UAM064 PROJECT:** a.exe, bookings.txt, event.cpp (selected), events.txt, reviews.txt.
 - OUTLINE:** No symbols found in document 'event.cpp'.
- Terminal Panel:**

```

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 105
Booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 3
Enter Event ID to cancel: 101
Booking Deleted!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 105
Enter Review: amazing
Review Added!

```

```

----- User MENU -----
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 5

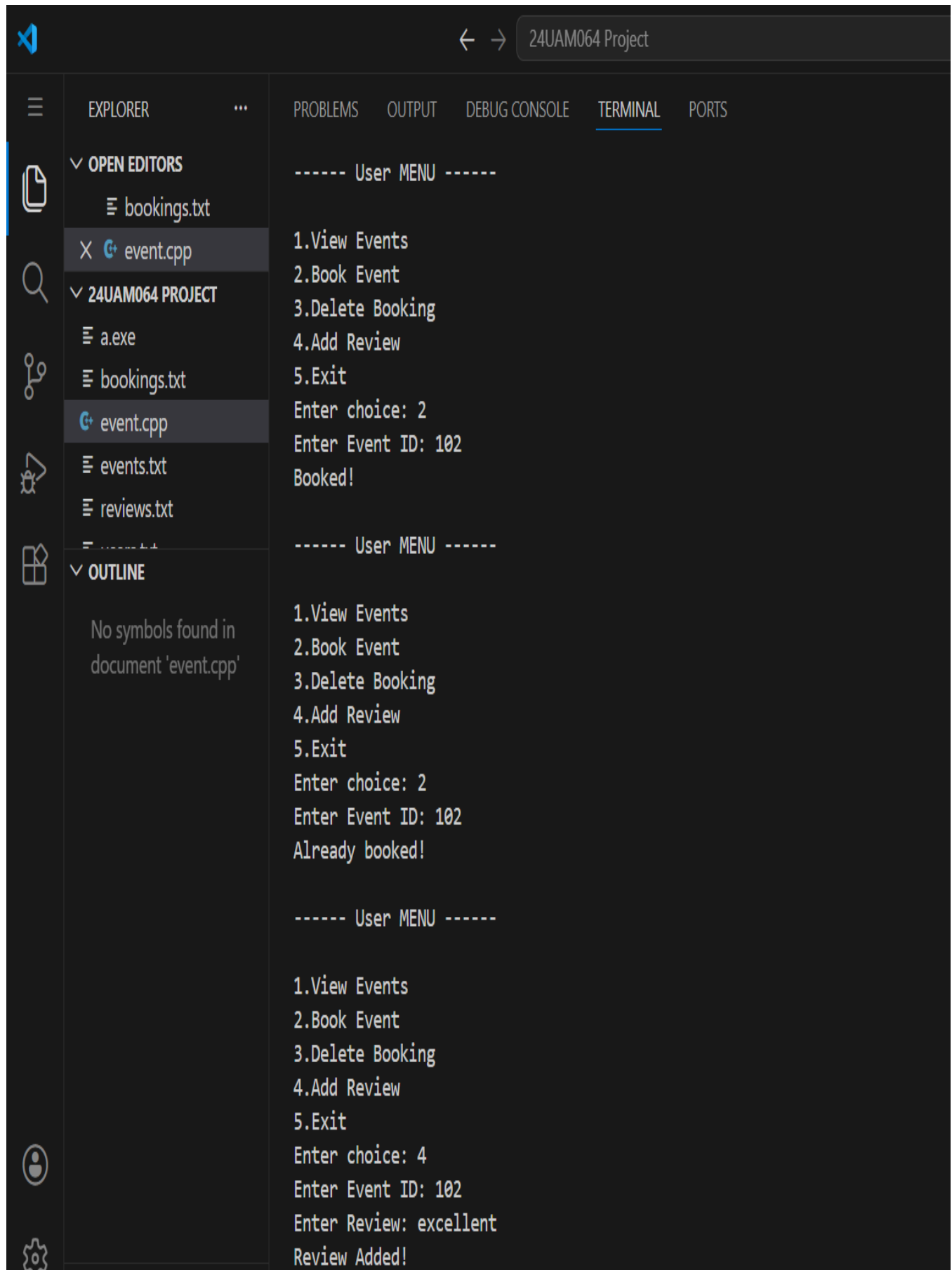
----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 3
Username: varad
Password: 5866
Login Success!

----- User MENU -----
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 1

      ID      NAME      TYPE      BUDGET  LOCATION  DATE
      101    codechef  technical    8000    kolhapur  06/08/2026
      102  fashionshow nontechnical 12000    sangli    14/06/2026
      104    championcup    sports    9000    tardal    04/05/2026
      105    salesgurunontechnical 23000    pune     23/07/2026

----- User MENU -----

```



Visual Studio Code interface showing the Explorer, Output, and Terminal panels.

EXPLORER

- OPEN EDITORS
 - bookings.txt
 - event.cpp
- 24UAM064 PROJECT
 - a.exe
 - bookings.txt
 - event.cpp
 - events.txt
 - reviews.txt
- OUTLINE
 - No symbols found in document 'event.cpp'

OUTPUT

```

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 102
Booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 102
Already booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 102
Enter Review: excellent
Review Added!

```

TERMINAL

```

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 102
Booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 2
Enter Event ID: 102
Already booked!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 102
Enter Review: excellent
Review Added!

```

```

24UAM064 Project
EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'
  TIMELINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

----- User MENU -----
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 5

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 3
Username: prajakta
Password: 3456
Login Success!

----- User MENU -----
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 1

      ID      NAME      TYPE      BUDGET      LOCATION      DATE
    101  codechef  technical      8000    kolhapur    06/08/2026
    102 fashionshow  nontechnical  12000    sangli     14/06/2026
    104 championcup    sports      9000    tardal     04/05/2026
    105 salesgurunontechnical  23000    pune      23/07/2026

----- User MENU -----

```

```
24UAM064 Project

EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'

TERMINAL
  ----- User MENU -----
  1.View Events
  2.Book Event
  3.Delete Booking
  4.Add Review
  5.Exit
  Enter choice: 2
  Enter Event ID: 101
  Booked!

  ----- User MENU -----
  1.View Events
  2.Book Event
  3.Delete Booking
  4.Add Review
  5.Exit
  Enter choice: 2
  Enter Event ID: 105
  Booked!

  ----- User MENU -----
  1.View Events
  2.Book Event
  3.Delete Booking
  4.Add Review
  5.Exit
  Enter choice: 2
  Enter Event ID: 104
  Booked!
```

Visual Studio Code interface showing the Event Management System project. The Explorer sidebar on the left shows the file structure with 'event.cpp' selected. The Terminal window on the right displays the program's output, including a menu and user interactions for deleting a booking and adding reviews.

```

24UAM064 Project

EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 3
Enter Event ID to cancel: 105
Booking Deleted!

----- User MENU -----

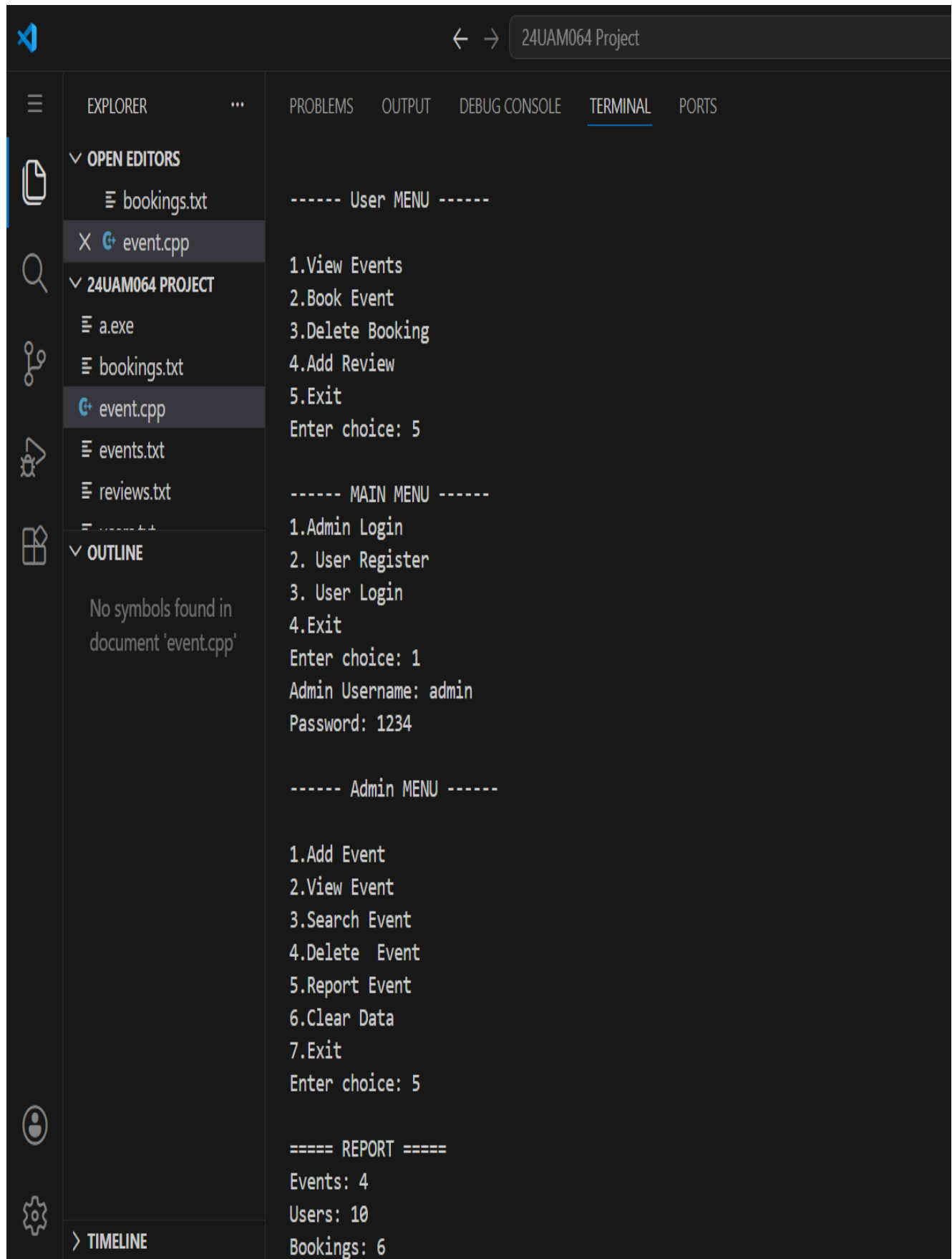
1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 104
Enter Review: good
Review Added!

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 4
Enter Event ID: 101
Enter Review: verygood
Review Added!

----- User MENU -----

```



```

24UAM064 Project

EXPLORER
  OPEN EDITORS
    bookings.txt
    event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

----- User MENU -----

1.View Events
2.Book Event
3.Delete Booking
4.Add Review
5.Exit
Enter choice: 5

----- MAIN MENU -----

1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 1
Admin Username: admin
Password: 1234

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 5

===== REPORT =====
Events: 4
Users: 10
Bookings: 6

```

```

24UAM064 Project
1 | 1 | v

EXPLORER
  OPEN EDITORS
    bookings.txt
    X event.cpp
  24UAM064 PROJECT
    a.exe
    bookings.txt
    event.cpp
    events.txt
    reviews.txt
  OUTLINE
    No symbols found in document 'event.cpp'

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 5

===== REPORT =====
Events: 4
Users: 10
Bookings: 6

----- Admin MENU -----

1.Add Event
2.View Event
3.Search Event
4.Delete Event
5.Report Event
6.Clear Data
7.Exit
Enter choice: 7

----- MAIN MENU -----
1.Admin Login
2. User Register
3. User Login
4.Exit
Enter choice: 4

```


CONCLUSION

The Event Management System is a comprehensive and user-friendly application developed to efficiently manage various aspects of event organization, including event creation, user registration, booking, and review management. The system successfully reduces manual work and provides a structured digital solution using file handling techniques.

Through this system, the administrator can easily add, view, search, and delete events, as well as generate reports and manage all stored data. On the other hand, users are provided with functionalities such as registration, login, event viewing, booking, cancellation, and submitting reviews. This ensures smooth interaction between the admin and users within the system.

The implementation of validation checks, such as duplicate event detection, future date verification, and booking restrictions, enhances the reliability and accuracy of the system. The use of separate files for storing events, users, bookings, and reviews ensures proper data organization and persistence.

Overall, the Event Management System demonstrates how basic programming concepts like file handling, conditional statements, loops, and functions can be effectively used to build a real-world application. It serves as a strong foundation for developing more advanced systems with database integration, graphical user interfaces, and enhanced security features in the future.